PROJECT REPORT · SEPTEMBER 2026

বাংলা ডিস্টিল

Bangla Distill — Bengali Language Model Fine-Tuning

Knowledge Distillation for Bangla NLP

Fine-tuning Qwen2.5-0.5B on synthetic Bengali corpora generated by large teacher models — enabling fluent Bengali text generation in a compact, portable adapter.

Qwen2.5-0.5B Base | QLoRA · 4-bit NF4 | LoRA rank=16 · α=32 | Gemma-2-9B-IT Teacher | Qwen-Large Teacher | TRL · PEFT · bitsandbytes | ~16.8 MB Adapter

5,985	2	375	16.8 MB
TRAINING EXAMPLES	TEACHER MODELS	TRAINING STEPS	FINAL ADAPTER SIZE

📋 TABLE OF CONTENTS

1. Executive Summary
2. Project Goals & Motivation
3. Knowledge Distillation Pipeline
4. Dataset & Training Corpus
5. Model Architecture & Configuration
6. Training Hyperparameters
7. Project File Structure
8. Inference & Deployment
9. Key Artifacts
10. Observations & Known Issues
11. Next Steps & Recommendations

1 Executive Summary

Bangla Distill is an applied NLP project that leverages *knowledge distillation* to transfer Bengali language capability from two large "teacher" models into a compact student model. The base student model — **Qwen/Qwen2.5-0.5B** (~500 million parameters) — is fine-tuned via **QLoRA** (Quantized Low-Rank Adaptation) on 5,985 teacher-generated Bengali articles.

The resulting LoRA adapter weighs only **~16.8 MB** and can be loaded onto a free-tier Google Colab T4 GPU alongside the 4-bit quantized base model. The project ships a ready-to-use Colab inference notebook and all training artifacts, making it immediately reproducible.

500M	5,985	16.8 MB
BASE MODEL PARAMETERS	TRAINING ARTICLES	ADAPTER FILE SIZE

15	2
SAVED CHECKPOINTS	TEACHER MODELS USED

2 Project Goals & Motivation

Bengali is one of the world's most widely spoken languages (~230 million speakers), yet it remains underrepresented in small, efficient language models. Large models like Gemma-2-9B or full Qwen models handle Bengali well, but require significant hardware — making them inaccessible for edge deployment, low-resource environments, or free-tier cloud compute.

✅ Primary Objectives

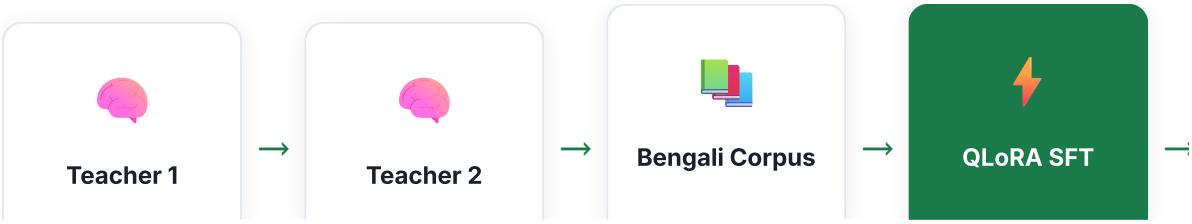
- Build a Bengali-capable model under 1B parameters
- Enable inference on a T4 GPU (free Colab tier)
- Use knowledge distillation to maximize quality
- Produce a portable, shareable LoRA adapter

🛠 Technical Approach

- Two diverse teacher models for corpus variety
- QLoRA: 4-bit NF4 quant + rank-16 LoRA adapters
- SFT via Hugging Face TRL's SFTTrainer
- Chat template for structured inference

3 Knowledge Distillation Pipeline

The project follows a classic *offline knowledge distillation* paradigm: teacher models generate high-quality text, which is saved as a static corpus, and the student model is then supervised on this corpus.



Gemma-2-9B-IT | Qwen (large) | 5,985 articles · JSONL | 375 steps · 4-bit NF4

Offline Distillation

Unlike online distillation (which aligns logits during training), this project uses the teachers' generated text as ground-truth supervision — simpler to implement and reproducible without the teachers being present at training time.

4 Dataset & Training Corpus

Corpus Overview

Both teacher models were prompted to generate informational Bengali articles on diverse topics (history, science, geography, culture, etc.). Their outputs were collected, deduplicated, and stored as JSONL files.

FILE	TEACHER MODEL	RECORDS	FILE SIZE	ROLE
`teacher_corpus_gemma.jsonl`	Gemma-2-9B-IT	3,000	2.7 MB	Full Gemm corpus
`teacher_corpus_gemma_part0.jsonl`	Gemma-2-9B-IT	~1,500	1.4 MB	Split – part 0
`teacher_corpus_gemma_part1.jsonl`	Gemma-2-9B-IT	~1,500	1.3 MB	Split – part 1
`teacher_corpus_qwen.jsonl`	Qwen (large)	2,985	3.1 MB	Full Qwen corpus
Total	**2 teachers**	**5,985**	**~5.8 MB (unique)**	**—**

Data Schema

Each JSONL record follows a consistent four-field schema:

```
{
  "id":      0,
  "title":   "বাংলাদেশ",          // Bengali article title
  "teacher": "gemma-2-9b-it",    // which teacher generated this
  "text":    "বাংলাদেশ দক্ষিণ ... "  // full Bengali article body
}
```

Corpus Composition

Gemma-2-9B-IT — 3,000 / 5,985 (50.1%)

Qwen (large) — 2,985 / 5,985 (49.9%)

> 💡 **Dual-Teacher Advantage**
>
> Using two distinct teacher models with different architectures and training data helps inject stylistic and factual diversity into the corpus, which typically improves the student model's generalization compared to single-teacher distillation.

5 Model Architecture & Configuration

Base Model

PROPERTY	VALUE
Model ID	`Qwen/Qwen2.5-0.5B`
Parameters	~500 million
Architecture	Decoder-only Transformer (Causal LM)
Source	HuggingFace Hub
Quantization at inference	4-bit NF4 (bitsandbytes)

LoRA Adapter Configuration

PARAMETER	VALUE	NOTES
PEFT type	`LORA`	Standard LoRA
LoRA rank (r)	16	Low-rank bottleneck dimension
LoRA alpha	32	Scaling: α/r = 2.0
LoRA dropout	0.05	Applied during training only
Bias	none	Bias terms not adapted
Task type	`CAUSAL_LM`	Autoregressive generation
DoRA	False	Standard LoRA, not Weight-Decomposed
RSLoRA	False	Standard rank-1 scaling
PEFT version	0.20.0	—

Target Modules

LoRA adapters are applied to all 7 attention and feed-forward projection matrices:

`q_proj` `k_proj` `v_proj` `o_proj` `gate_proj` `up_proj` `down_proj`

Targeting all projection layers (not just attention) is a more aggressive adaptation strategy that typically yields better fine-tuning performance at the cost of a slightly larger adapter.

6 Training Hyperparameters

HYPERPARAMETER	VALUE
Training method	SFT (SFTTrainer · TRL)
Epochs	1
Effective batch size	8

QUANTIZATION SETTING	VALUE
Quantization type	4-bit NF4
Compute dtype	FP16
AMP (Automatic Mixed Precision)	Disabled

HYPERPARAMETER	VALUE
Total steps	375
Checkpoint freq.	Every 25 steps
Checkpoints saved	15

QUANTIZATION SETTING	VALUE
BF16	Disabled
Double quantization	False

Framework Versions

LIBRARY	VERSION
TRL	1.12.0
Transformers	5.16.1
PyTorch	2.10.0+cu128
Datasets	5.0.1
Tokenizers	0.23.1
PEFT	0.20.0
bitsandbytes	latest (via pip)

Training Progress

Checkpoints were saved at regular 25-step intervals across the full training run:

STEP 25 · STEP 50 · STEP 75 · STEP 100 · STEP 125 · STEP 150 · STEP 175
STEP 200 · STEP 225 · STEP 250 · STEP 275 · STEP 300 · STEP 325
STEP 350 · STEP 375 ✓ FINAL

7 Project File Structure

```
Bangla Distill/
├── run_bangla_student.ipynb ← Google Colab inference notebook
├── teacher_corpus_gemma.jsonl 2.7 MB · 3,000 records
├── teacher_corpus_gemma_part0.jsonl 1.4 MB · split part 0
├── teacher_corpus_gemma_part1.jsonl 1.3 MB · split part 1
├── teacher_corpus_qwen.jsonl 3.1 MB · 2,985 records
├── qwen-2.5-0.5b-bangla-complete.zip ~484 MB distributable bundle
└── qwen-2.5-0.5b-bangla-complete/
    ├── README.txt ← Package summary
    ├── final-lora-adapter/ ← Final trained weights
    │   ├── adapter_config.json 1.1 KB · PEFT config
    │   ├── adapter_model.safetensors 16.8 MB · trained weights
    │   ├── tokenizer.json 10.9 MB · full tokenizer
    │   ├── tokenizer_config.json 697 B
    │   ├── chat_template.jinja 2.4 KB · prompt format
    │   ├── TRAINING_INFO.txt ← Hyperparameter summary
    │   └── README.md ← HF model card (template)
    └── training-output/ ← Full SFTTrainer output
        ├── README.md ← Auto-generated model card
        ├── checkpoint-25/
        ├── checkpoint-50/
        ├── ... (15 checkpoints total)
        └── checkpoint-375/ ← Final checkpoint
```

8 Inference & Deployment

The project ships a fully self-contained Google Colab notebook (`run_bangla_student.ipynb`) that walks users through 7 steps from a blank runtime to a running interactive Bengali chatbot.

STEP	ACTION	DETAILS
1	Install dependencies	`transformers`, `peft`, `accelerate`, `bitsandbytes`, `sentencepiece`
2	Upload adapter files	Upload all 5 files from `final-lora-adapter/`
3	Load base model + LoRA	4-bit NF4 quantized base, then `PeftModel.from_pretrained()`
4	Define generation helper	`generate_bangla()` with chat template
5	Run example prompts	Bengali history, AI questions — pre-written examples

STEP	ACTION	DETAILS
6	Interactive chat loop	Accept user Bengali input, generate responses in a loop
7	Base vs fine-tuned comparison	Disable adapter to compare raw vs distilled output

Generation Parameters

PARAMETER	DEFAULT VALUE	DESCRIPTION
max_new_tokens	300	Maximum tokens to generate per response
temperature	0.7	Controls output randomness (0 = deterministic)
top_p	0.9	Nucleus sampling — top 90% probability mass
repetition_penalty	1.1	Discourages repeated tokens/phrases
do_sample	True	Stochastic sampling (not greedy decoding)

Hardware Requirements

Minimum (Inference)

- Google Colab free tier (T4 GPU)
- ~5 GB VRAM (4-bit quantized model)
- ~2 GB RAM for tokenizer + adapter

Recommended (Inference)

- T4 or A100 (Colab Pro)
- Faster generation on A100 (bfloat16)
- Enables larger max_new_tokens

9 Key Artifacts

ARTIFACT	FILE	SIZE	PURPOSE
Inference Notebook	`run_bangla_student.ipynb`	9 KB	End-to-end Colab demo + interactive chat
Trained Adapter Weights	`adapter_model.safetensors`	16.8 MB	The core fine-tuned parameters
LoRA Config	`adapter_config.json`	1.1 KB	PEFT metadata & hyperparameters
Tokenizer	`tokenizer.json`	10.9 MB	Full BPE tokenizer (may include Bengali tokens)
Chat Template	`chat_template.jinja`	2.4 KB	Jinja2 prompt template for chat format
Gemma Corpus	`teacher_corpus_gemma.jsonl`	2.7 MB	3,000 Bengali articles from Gemma-2-9B-IT
Qwen Corpus	`teacher_corpus_qwen.jsonl`	3.1 MB	2,985 Bengali articles from Qwen-large
Full Package	`qwen-2.5-0.5b-bangla-complete.zip`	~484 MB	Complete distributable archive with all checkpoints

10 Observations & Known Issues

Unfilled Model Card

The `final-lora-adapter/README.md` is an auto-generated Hugging Face model card template. Nearly all sections contain "More Information Needed" placeholders. This should be filled in before publishing.

Placeholder in Training Output README

The auto-generated `training-output/README.md` contains `model="None"` in the quick-start snippet. This must be replaced with the actual model path or

HuggingFace Hub ID before sharing.

💡 **Single Epoch Training**

Only 1 epoch was performed over 5,985 examples. While QLoRA is sample-efficient, a second or third epoch typically yields measurable gains in language quality, especially for low-resource language fine-tuning.

ℹ️ **No Formal Evaluation**

There are no evaluation scores (BLEU, ROUGE, perplexity) reported. The notebook includes a qualitative base-vs-fine-tuned comparison (Step 7), but no quantitative benchmarking was performed.

🔴 **AMP Disabled**

Automatic Mixed Precision (AMP) and BF16 are both disabled. This may have slowed training — enabling BF16 on compatible hardware (A100/H100) could reduce training time by 30–50% without accuracy loss.

11 Next Steps & Recommendations

- [ ] **Complete the model card** — Fill in `final-lora-adapter/README.md` with model details, training data description, language info, and license.
- [ ] **Fix the training README placeholder** — Replace `model="None"` with the actual model identifier in `training-output/README.md`.
- [ ] **Run formal evaluation** — Compute perplexity on a held-out Bengali test set, or evaluate against Bengali NLP benchmarks (e.g., BanglaLM, BengaliNLP datasets).
- [ ] **Multi-epoch training** — Re-train with 2–3 epochs to improve fluency. Monitor validation loss to avoid overfitting on the 5,985-example corpus.
- [ ] **Expand the corpus** — Generate additional Bengali articles from the teacher models, or incorporate Bengali Wikipedia excerpts as additional supervised signal.
- [ ] **Enable BF16 / AMP** — Re-run training with `bf16=True` on A100 to speed up future training runs significantly.

- [ ] **Push to Hugging Face Hub** — Publish the adapter under a descriptive repo name (e.g., `your-username/qwen2.5-0.5b-bangla-lora`) for public access.
- [ ] **Merge and export** — Optionally merge the LoRA adapter into the base weights and export a standalone model for deployment without PEFT dependency.
- [ ] **Online distillation** — For a future iteration, explore online KD by aligning soft logits from the teacher during training, which typically outperforms offline SFT distillation.
